

### BASIC CONCEPTS OF C++ PROGRAMMING

#### C++ PROGRAMMING:

C++ is the most used and most popular programming language developed by Bjarne Stroustrup. C++ is a high-level and object-oriented programming language. This language allows developers to write clean and efficient code for large applications and software development, game development, and operating system programming.

#### FEATURES OF C++:

C++ is a general-purpose programming language that was developed as an enhancement of the C language to include an object-oriented paradigm. It is an imperative and **compiled** language. C++ has a number of features, including:

- Object-Oriented Programming
- Machine Independent
- Simple
- High-Level Language
- Popular
- Case-sensitive
- Compiler Based
- Dynamic Memory Allocation
- Memory Management
- Multi-threading

#### HELLO WORLD IN C++:

The Hello World Program in C++ is the basic program that is used to demonstrate how the coding process works.

```
#include <iostream.h>
```

```
int main()
```

```
{
```

```
    // Prints hello world
```

```
    cout << "Hello World";
```

```
    return 0;  
}
```

### Output:

Hello World

### Working of Hello World Program in C++

#### 1. // C++ program to display “Hello World”

This line is a comment line. A comment is used to display additional information about the program. A comment does not contain any programming logic.

#### 2. #include

This is a preprocessor directive. The **#include** directive tells the compiler to include the content of a file in the source code.

For example, **#include<iostream>** tells the compiler to include the standard iostream file which contains declarations of all the standard input/output library functions.

#### 3. int main() { }

A function is a group of statements that are designed to perform a specific task. The main() function is the entry point of every C++ program, no matter where the function is located in the program.

The opening braces ‘{’ indicates the beginning of the main function and the closing braces ‘}’ indicates the ending of the main function.

#### 4. cout<<“Hello World”;

std::cout is an instance of the std::ostream class, that is used to display output on the screen. Everything followed by the character << in double quotes ” ” is displayed on the output device. The semi-colon character at the end of the statement is used to indicate that the statement is ending there.

## 5. return 0

This statement is used to return a value from a function and indicates the finishing of a function. This statement is basically used in functions to return the results of the operations performed by a function.

### 1.4 STRUCTURE OF C++:

#### Structure of C++ Program

|               |   |                                                            |                         |
|---------------|---|------------------------------------------------------------|-------------------------|
|               | 1 | <code>#include &lt;iostream&gt;</code>                     | Header File             |
|               | 2 | <code>using namespace std;</code>                          | Standard Namespace      |
|               | 3 | <code>int main()</code>                                    | Main Function           |
| FUNCTION BODY | 4 | <code>{</code>                                             |                         |
|               | 5 | <code>int num1 = 24;</code><br><code>int num2 = 34;</code> | Declaration of Variable |
|               | 6 | <code>int result = num1 + num2;</code>                     | Expressions             |
|               | 7 | <code>cout &lt;&lt; result &lt;&lt; endl;</code>           | Output                  |
|               | 8 | <code>return 0;</code>                                     | Return Statement        |
|               | 9 | <code>}</code>                                             |                         |

The image above shows the basic C++ program that contains header files, main function, namespace declaration, etc. Let's try to understand them one by one.

### 1. Header File

The header files contain the definition of the functions and macros we are using in our program. They are defined on the top of the C++ program.

In **line #1**, we used the **#include <iostream>** statement to tell the compiler to include an **iostream** header file library which stores the definition of the cin and cout methods that we have used for input and output. **#include** is a preprocessor directive using which we import header files.

#### Syntax:

```
#include <library_name>
```

To know more about header files, please refer to the article – [Header Files in C/C++](#).

### 2. Namespace

A namespace in C++ is used to provide a scope or a region where we define identifiers. It is used to avoid name conflicts between two identifiers as only unique names can be used as identifiers.

In **line #2**, we have used the **using namespace std** statement for specifying that we will be the standard namespace where all the standard library functions are defined.

### Syntax:

```
using namespace std;
```

### 3. Main Function

Functions are basic building blocks of a C++ program that contains the instructions for performing some specific task. Apart from the instructions present in its body, a function definition also contains information about its return type and parameters. To know more about C++ functions, please refer to the article [Functions in C++](#).

In **line #3**, we defined the main function as **int main()**. The main function is the most important part of any C++ program. The program execution always starts from the main function. All the other functions are called from the main function. In C++, the main function is required to return some value indicating the execution status.

### Syntax:

```
int main() {  
  
    ... code ....  
  
    return 0;  
  
}
```

### 4. Blocks

Blocks are the group of statements that are enclosed within { } braces. They define the scope of the identifiers and are generally used to enclose the body of functions and control statements.

The body of the main function is from **line #4** to **line #9** enclosed within { }.

**Syntax:**

```
{  
  
    // Body of the Function  
  
    return 0;  
}
```

### 5. Semicolons

As you may have noticed by now, each statement in the above code is followed by a ( ; ) semicolon symbol. It is used to terminate each line of the statement of the program. When the compiler sees this semicolon, it terminates the operation of that line and moves to the next line.

**Syntax:**

*any\_statement ;*

### 6. Identifiers

We use identifiers for the naming of variables, functions, and other user-defined data types. An identifier may consist of uppercase and lowercase alphabetical characters, underscore, and digits. The first letter must be an underscore or an alphabet.

**Example:**

```
int num1 = 24;
```

```
int num2 = 34;
```

num1 & num2 are the identifiers and int is the data type.

### 7. Keywords

In the C++ programming language, there are some reserved words that are used for some special meaning in the C++ program. It can't be used for identifiers.

For example, the words **int**, **return**, and **using** are some keywords used in our program. These all have some predefined meaning in the C++ language.

There are total 95 keywords in C++. These are some keywords.

int      void      if      while      for      auto      bool      break

this      static      new      true      false      case      char      class

## **8. Basic Output cout**

In **line #7**, we have used the **cout** method which is the basic output method in C++ to output the sum of two numbers in the standard output stream (stdout).

### **Syntax:**

```
cout << result << endl;
```

### **COMMENTS:**

Comments in C++ are used to summarize an algorithm, identify a variable's purpose, or clarify a code segment that appears unclear.

#### **Types of Comments in C++**

- **Single-line comment**
- **Multi-line comment**

#### **1. Single Line Comment**

In C++ Single line comments are represented as // double forward slash. It applies comments to a single line only. The compiler ignores any text after // and it will not be executed.

##### **Syntax:**

```
// Single line comment
```

#### **2. Multi-Line Comment**

A multi-line comment can occupy many lines of code, it starts with /\* and ends with \*/, but it cannot be nested. Any text between /\* and \*/ will be ignored by the compiler.

##### **Syntax:**

```
/*  
    Multiline Comment  
    .  
    .  
    .  
*/
```

### TOKENS IN C++

Tokens can be defined as the smallest building block of C++ programs that the compiler understands. The types of tokens in C++ are:

- Identifiers
- Keywords
- Constants
- Strings
- Special Symbols
- Operators

### IDENTIFIERS:

In C++, entities like variables, functions, classes, or structs must be given unique names within the program so that they can be uniquely identified. The unique names given to these entities are known as identifiers.

#### Rules for create identifiers:

An identifier can only begin with a **letter or an underscore(\_)**.

An identifier can consist of **letters** (A-Z or a-z), **digits** (0-9), and **underscores** (\_). White spaces and Special characters can not be used as the name of an identifier.

Keywords cannot be used as an identifier because they are reserved words to do specific tasks. For example, **string**, **int**, **class**, **struct**, etc.

Identifier must be **unique** in its namespace.

### KEYWORDS:

Keywords in C++ are the tokens that are the reserved words in programming languages that have their specific meaning and functionalities within a program. Keywords cannot be used as an identifier to name any variables.

For example, a variable or function cannot be named as 'int' because it is reserved for declaring integer data type.

**There are 95 keywords reserved in C++. Some of the main Keywords are:**

|       |     |       |      |       |       |          |
|-------|-----|-------|------|-------|-------|----------|
| break | try | catch | char | class | const | continue |
|-------|-----|-------|------|-------|-------|----------|

|           |             |          |          |           |        |        |
|-----------|-------------|----------|----------|-----------|--------|--------|
| default   | delete      | auto     | else     | friend    | for    | float  |
| long      | new         | operator | private  | protected | public | return |
| short     | sizeof      | static   | this     | typedef   | enum   | throw  |
| mutable   | struct      | case     | register | switch    | and    | or     |
| namespace | static_cast | goto     | not      | xor       | bool   | do     |
| double    | int         | unsigned | void     | virtual   | union  | while  |

**CONSTANTS:**

Constants are the tokens in C++ that are used to define variables at the time of initialization and the assigned value cannot be changed after that.

**Syntax**

```
const data_type variable_name = value;
```

**Example:**

```
const int n=4;
```

**STRINGS:**

A **string** is not a built-in data type like 'int', 'char', or 'float'. It is a class available in the STL library which provides the functionality to work with a sequence of characters, that represents a string of text.

**Syntax:**

```
string variable_name;
```

**Example:**

```
String str= "Hello";
```

**SPECIAL SYMBOLS:**

Special symbols are the token characters having specific meanings within the syntax of the programming language.

- **Semicolon (;):** It is used to terminate the statement.

- **Square brackets []:** They are used to store array elements.
- **Curly Braces {}:** They are used to define blocks of code.
- **Scope resolution (::):** Scope resolution operator is used to access members of namespaces, classes, etc.
- **Dot (.):** Dot operator also called member access operator used to access class and struct members.
- **Assignment operator '=':** This operator is used to assign values to variables.
- **Double-quote ("):** It is used to enclose string literals.
- **Single-quote ('):** It is used to enclose character literals.

### OPERATORS:

C++ operators are special symbols that are used to perform operations on operands such as variables, constants, or expressions.

#### Types of Operators

- Unary Operators
- Binary Operators
- Ternary Operators

#### 1. Unary Operators

Unary operators are used with single operands only. They perform the operations on a single variable. For example, increment and decrement operators.

**Increment operator ( ++ ):** It is used to increment the value of an operand by 1.

**Decrement operator ( — ):** It is used to decrement the value of an operand by 1.

#### 2. Binary Operators

They are used with the two operands and they perform the operations between the two variables. For example (A<B), less than (<) operator compares A and B, returns true if A is less than B else returns false.

**Arithmetic Operators:** These operators perform basic arithmetic operations on operands. They include '+', '-', '\*', '/', and '%'

**Comparison Operators:** These operators are used to compare two operands and they include ‘==’, ‘!=’, ‘<’, ‘>’, ‘<=’, and ‘>=’.

**Logical Operators:** These operators perform logical operations on boolean values. They include ‘&&’, ‘||’, and ‘!’.

**Assignment Operators:** These operators are used to assign values to variables and they include ‘variables’, ‘-=’, ‘\*=’, ‘/=’, and ‘%=’.

**Bitwise Operators:** These operators perform bitwise operations on integers. They include ‘&’, ‘|’, ‘^’, ‘~’, ‘<<’, and ‘>>’.

### 3. Ternary Operator

The ternary operator is the only operator that takes three operands. It is also known as a conditional operator that is used for conditional expressions.

#### Syntax:

Expression1 ? Expression2 : Expression3;

#### DIFFERENCE BETWEEN KEYWORD AND IDENTIFIERS:

| SR. NO. | KEYWORD                                                                                                                          | IDENTIFIER                                                                                                                                                                |
|---------|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | Keywords are predefined word that gets reserved for working program that have special meaning and cannot get used anywhere else. | Identifiers are the values used to define different programming items such as variables, integers, structures, unions and others and mostly have an alphabetic character. |
| 2       | Specify the type/kind of entity.                                                                                                 | Identify the name of a particular entity.                                                                                                                                 |
| 3       | It always starts with a lowercase letter.                                                                                        | First character can be a uppercase, lowercase letter or underscore.                                                                                                       |
| 4       | A keyword should be in lower case.                                                                                               | An identifier can be in upper case or lower case.                                                                                                                         |
| 5       | A keyword contains only alphabetical characters.                                                                                 | An identifier can consist of alphabetical characters, digits and underscores.                                                                                             |
| 6       | They help to identify a specific property that exists within a computer language.                                                | They help to locate the name of the entity that gets defined along with a keyword.                                                                                        |
| 7       | No special symbol, punctuation is used.                                                                                          | No punctuation or special symbol except ‘underscore’ is used.                                                                                                             |
| 8       | Examples of keywords are: int, char, if, while, do, class etc.                                                                   | Examples of identifiers are: Test, count1, high_speed, etc.                                                                                                               |

**VARIABLES**

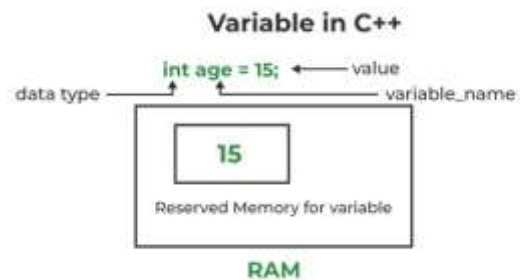
Variables in C++ is a name given to a memory location. It is the basic unit of storage in a program. The value stored in a variable can be changed during program execution.

Syntax:

type variable\_name;

Example:

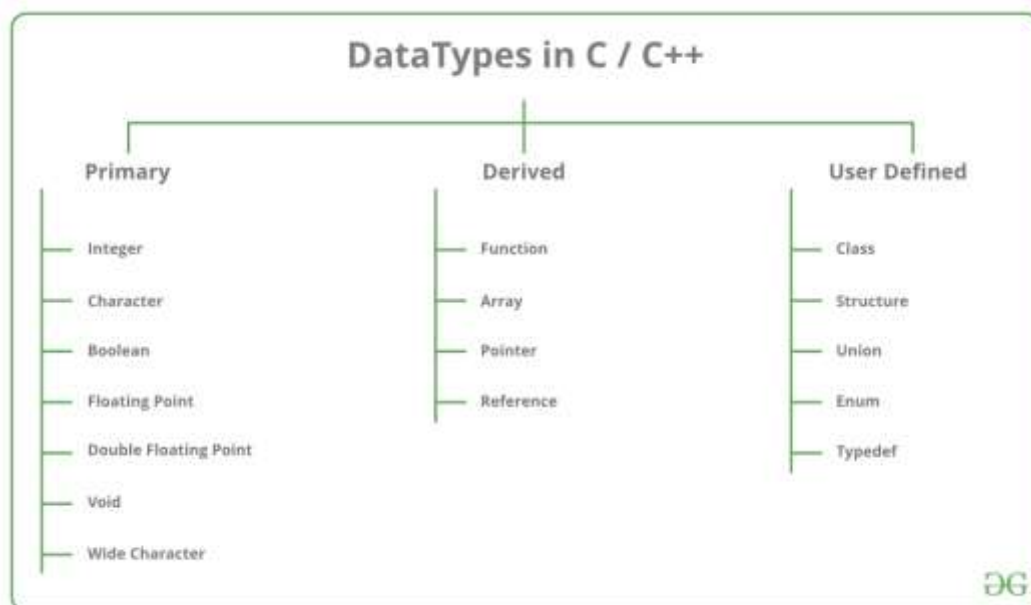
int a;

**DATA TYPES:**

The data types are used to tell the variables the type of data they can store.

Types of data types:

1. Primary or Built-in or Fundamental data type
2. Derived data types
3. User-defined data types



**1. Primitive Data Types:** These data types are built-in or predefined data types and can be used directly by the user to declare variables. example: int, char, float, bool, etc. Primitive data types available in C++ are:

- Integer
- Character
- Boolean
- Floating Point
- Double Floating Point
- Valueless or Void
- Wide Character

**2. Derived Data Types:** Derived data types that are derived from the primitive or built-in datatypes are referred to as Derived Data Types. These can be of four types namely:

- Function
- Array
- Pointer
- Reference

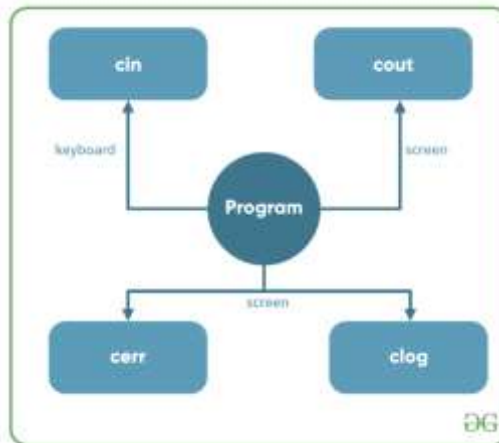
**3. Abstract or User-Defined Data Types:** Abstract or User-Defined data types are defined by the user itself. Like, defining a class in C++ or a structure. C++ provides the following user-defined datatypes:

- Class
- Structure
- Union
- Enumeration
- Typedef defined Datatype

### INPUT AND OUTPUT IN C++

C++ comes with libraries that provide us with many ways for performing input and output. In C++ input and output are performed in the form of a sequence of bytes or more commonly known as **streams**.

- **Input Stream:** If the direction of flow of bytes is from the device(for example, Keyboard) to the main memory then this process is called input.
- **Output Stream:** If the direction of flow of bytes is opposite, i.e. from main memory to device( display screen ) then this process is called output.



### CIN IN C++

The cin object in C++ is an object of class istream. It is used to accept the input from the standard input device i.e. keyboard.

### cout IN C++:

The cout object in C++ is an object of class ostream. It is defined in ostream header file. It is used to display the output to the standard output device i.e. monitor.

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
// Driver Code
```

```
int main()
```

```
{
```

```
    string name;
```

```
    int age;
```

```
    // Take multiple input using cin
```

```
    cin >> name >> age;
```

```
    // Print output
```

```
    cout << "Name : " << name << endl;
```

```
    cout << "Age : " << age << endl;
```

```
    return 0;
}
```

Output:

Siva

26

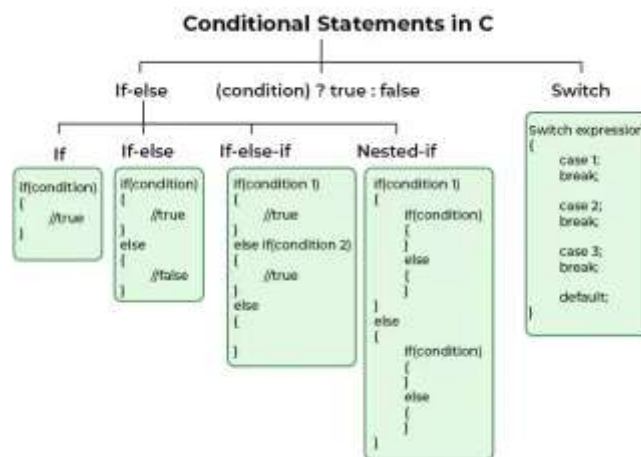
Name : Siva

Age : 26

### CONDITIONAL STATEMENT:

The **conditional statements** (also known as decision control structures) such as if, if else, switch, etc. are used for decision-making purposes in C programs.

They are also known as Decision-Making Statements and are used to evaluate one or more conditions and make the decision whether to execute a set of statements or not.



### SIMPLE IF

The **if** statement is the simplest of the three and is used to run a certain piece of code only if a certain condition is true. For example:

```
int x = 5;
if (x == 5) {
    cout << "x is 5" << endl;
}
```

### IF...ELSE

The **if-else** statement is used when we want to execute some code only if some condition exists. If the given condition is true then code will be executed otherwise else statement will be used to run other part of the code. For example:

```
int x = 5;
if (x == 5) {
    cout << "x is 5" << endl;
} else {
    cout << "x is not 5" << endl;
}
```

### IF...ELSE..IF:

```
if (x > 0)
    cout << "x is positive";
else if (x < 0)
    cout << "x is negative";
else
    cout << "x is 0";
```

### SWITCH

The **switch** statement is used to execute different blocks of code based on the value of a variable. For example:

```
int x = 2;
switch (x) {
    case 1:
        cout << "x is 1" << endl;
        break;
    case 2:
        cout << "x is 2" << endl;
        break;
    default:
```

```

        cout << "x is not 1 or 2" << endl;
        break;
    }

```

### NESTED-IF

Nested if-else statements are those statements in which there is an if statement inside another if else.

```

#include <iostream>
using namespace std;
int main()
{
    // declaring three numbers
    int a = 10;
    int b = 2;
    int c = 6;

    // outermost if else
    if (a < b) {
        // nested if else
        if (c < b) {
            printf("%d is the greatest", b);
        }
        else {
            printf("%d is the greatest", c);
        }
    }
    else {
        // nested if else
        if (c < a) {
            printf("%d is the greatest", a);
        }
        else {
            printf("%d is the greatest", c);
        }
    }

    return 0;
}

```

## LOOPING STATEMENT:

There are three types of loops used in C++ programming:

1. for loop
2. while loop
3. do while loop

### 1.) for loop in C++

A for loop is a control structure that enables a set of instructions to get executed for a specified number of iterations. It is an **entry-controlled** loop.

#### Syntax

```
for(initialization; testcondition; updateExpression)
{
    //code to be executed
}
```

#### Example

// Program to print numbers from 1 to 10

```
#include <iostream>
using namespace std;
int main() {
    for (int i = 1; i <= 10; i++) {
        cout << i << endl;
    }
    return 0;
}
```

#### Output

### 2.) Nested for loop in C++

It is any type of **loop** inside a for loop.

#### Example

```
#include <iostream>
```

```
using namespace std;
int main() {
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            cout << j << " ";
        }
        cout << endl;
    }
    return 0;
}
```

## Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

## 3.) Infinite for loop in C++

An infinite for loop gets created if the for loop condition **remains** true, or if the programmer **forgets** to apply the test condition/terminating statement within the for loop statement.

Syntax

```
for( ; )
{
    // body of the for loop.
}
```

In the above syntax, there is **no condition** hence, this loop will execute **infinite times**.

Example

```
#include <iostream>

int main() {
    for (;;) {
```

```
std::cout << "Hello World" << std::endl;  
}  
return 0;  
}
```

In the above code, we have run the for loop **infinite** times, so "**Hello World**" will be displayed infinitely.

Output

```
Hello World  
Hello World  
Hello World  
Hello World  
Hello World  
...
```

#### **4.) for-each loop/ range-based for loop**

This is a new kind of loop introduced in the C++11 version. It is used **exclusively to traverse** through elements in an array and vector.

Syntax

```
for (variableName : arrayName/vectorName) {  
    // code block to be executed  
}
```

Here, for every value in the array or vector, the for loop is executed and the value is assigned to the variable.

Example

```
#include <iostream>  
using namespace std;  
int main() {  
    int num[7] = {10, 20, 30, 40, 50, 60, 70};  
    for (int i : num) {  
        cout << i << "\n";  
    }
```

```
}  
return 0;  
}
```

Output

```
10  
20  
30  
40  
50  
60  
70
```

### **WHILE LOOPS:**

A while loop evaluates the condition. If the condition evaluates to true, the code inside the while loop is executed. The condition is evaluated again. This process continues until the condition is false. When the condition evaluates to condition, the loop terminates.

```
While(condition){  
  
    //body of the loop  
  
}
```

Example:

```
// C++ Program to print numbers from 1 to 5  
#include <iostream.h>  
using namespace std;  
int main() {  
    int i = 1;  
    // while loop from 1 to 5  
    while (i <= 5) {  
        cout << i << " ";  
        ++i;  
    }  
    return 0;  
}
```

```
}
```

Output:

1 2 3 4 5

**DO... WHILE.. LOOP:**

The do...while loop is a variant of the while loop with one important difference: the body of do...while loop is executed once before the condition is checked.

Its syntax is:

```
do {  
    // body of loop;  
}  
while (condition);
```

Example:

```
#include <iostream.h>  
using namespace std;  
int main() {  
    int i = 1;  
    // do...while loop from 1 to 5  
    do {  
        cout << i << " ";  
        ++i;  
    }  
    while (i <= 5);  
    return 0;  
}
```

Output:

1 2 3 4 5

**TYPE CONVERSION:**

Type conversion is the process that converts the predefined data type of one variable into an appropriate data type. Type conversion can be done in two ways in C++

- Implicit type conversion
- Explicit type conversion

**Implicit type conversion:**

The implicit type conversion is the type of conversion done automatically by the compiler without any human effort. It means an implicit conversion automatically converts one data type into another type based on some predefined rules of the C++ compiler. Hence, it is also known as the automatic type conversion.

Order for implicit conversion is as follows

**bool -> char -> short int -> int -> unsigned int -> long int -> unsigned long int -> long long int -> float -> double -> long double**

Program to convert int to float type using implicit type conversion

**Program1.cpp**

```
#include <iostream>
using namespace std;
int main ()
{
    // assign the integer value
    int num1 = 25;
    // declare a float variable
    float num2;
    // convert int value into float variable using implicit conversion
    num2 = num1;
    cout << " The value of num1 is: " << num1 << endl;
    cout << " The value of num2 is: " << num2 << endl;
    return 0;
}
```

**Output**

The value of num1 is: 25

The value of num2 is: 25

### Explicit conversion:

Conversions that require **user intervention** to change the data type of one variable to another, is called the **explicit type conversion**.

Example:

```
#include <iostream>

using namespace std;

int main ()
{
    float f2 = 6.7;
    // use cast operator to convert data from one type to another
    int x = static_cast <int> (f2);
    cout << " The value of x is: " << x;
    return 0;
}
```

Output:

The value of x is: 6

**OBJECT ORIENTED PROGRAMMING IN C++**

**UNIT I - INTRODUCTION TO C++**

**SYLLABUS**

**Object Oriented Programming Concepts:** Complexity in software - The need for object-orientation – Abstraction – Encapsulation – Modularity – Hierarchy.

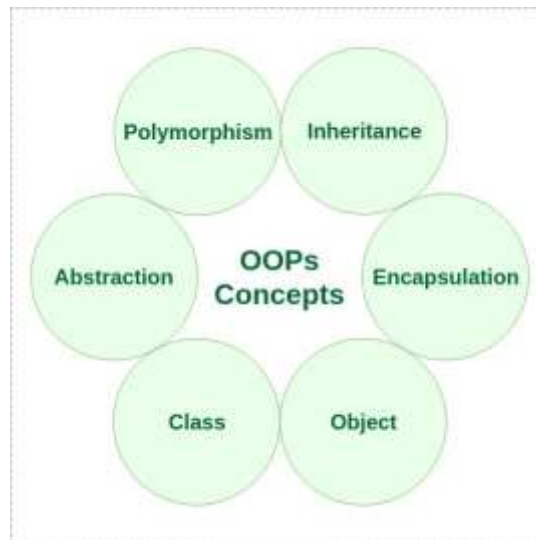
**Basic Elements of C++:** Classes – Objects – Data members and member functions – *private* and *public* access specifiers - Static members - Constructors – Singleton class - Destructors - Friend Functions and Friend Classes - Array of objects – Pointer to objects - *this* pointer – References – Dynamic memory allocation - Namespaces.

**OBJECT ORIENTED PROGRAMMING:**

Object-oriented programming – As the name suggests uses objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc. in programming. The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except that function.

There are some basic concepts that act as the building blocks of OOPs i.e.

1. Class
2. Objects
3. Encapsulation
4. Abstraction
5. Polymorphism
6. Inheritance
7. Dynamic Binding
8. Message Passing



### CLASS:

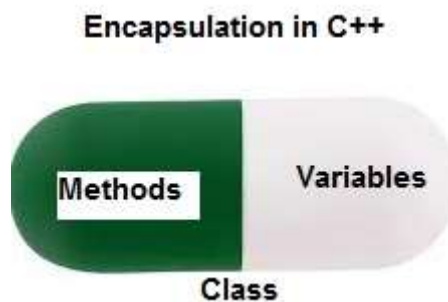
- A Class is a user-defined data type that has data members and member functions.
- Data members are the data variables and member functions are the functions used to manipulate these variables together these data members and member functions define the properties and behavior of the objects in a Class.

### OBJECT:

An Object is an identifiable entity with some characteristics and behavior. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

### ENCAPSULATION

Encapsulation is defined as wrapping up data and information under a single unit. In Object-Oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them.



### ABSTRACTION:

Data abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

### **POLYMORPHISM:**

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. A person at the same time can have different characteristics.

C++ supports operator overloading and function overloading.

- **Operator Overloading:** The process of making an operator exhibit different behaviors in different instances is known as operator overloading.
- **Function Overloading:** Function overloading is using a single function name to perform different types of tasks. Polymorphism is extensively used in implementing inheritance.

### **INHERITANCE:**

The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.

- **Sub Class:** The class that inherits properties from another class is called Sub class or Derived Class.
- **Super Class:** The class whose properties are inherited by a sub-class is called Base Class or Superclass.
- **Reusability:** Inheritance supports the concept of “reusability”

### **DYNAMIC BINDING:**

In dynamic binding, the code to be executed in response to the function call is decided at runtime. C++ has virtual functions to support this.

### **MESSAGE PASSING:**

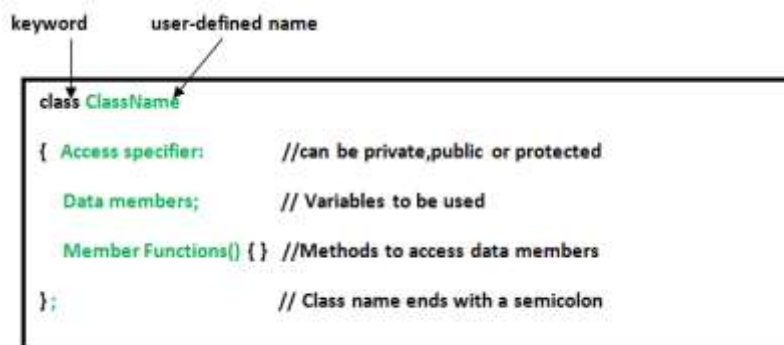
Objects communicate with one another by sending and receiving information. A message for an object is a request for the execution of a procedure and therefore will invoke a function in the receiving object that generates the desired results.

### CLASS:

A Class is a user-defined data type that has data members and member functions.

Data members are the data variables and member functions are the functions used to manipulate these variables together these data members and member functions define the properties and behavior of the objects in a Class.

Syntax:



Example:

```
class Student {
    public:
        int id;//data member (also instance variable)
        string name;//data member(also instance variable)
};
```

### OBJECT:

An Object is an identifiable entity with some characteristics and behavior. An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Syntax:

```
ClassName ObjectName;
```

Example for Class and Object:

```
#include <iostream.h>
using namespace std;
class Student {
```

```
public:
    int id;//data member (also instance variable)
    string name;//data member(also instance variable)
};
int main() {
    Student s1; //creating an object of Student
    s1.id = 201;
    s1.name = "Sonoo Jaiswal";
    cout<<s1.id<<endl;
    cout<<s1.name<<endl;
    return 0;
}
```

Output:

201 Sonoo 990000

202 Nakul 29000

### **CONSTRUCTORS:**

Constructors are special class members which are called by the compiler every time an object of that class is instantiated. Constructors have the same name as the class and may be defined inside or outside the class definition. There are 3 types of constructors:

- Default Constructors
- Parameterized Constructors
- Copy Constructors

#### **Syntax:**

```
Class_Name(list of parameters){
//constructor definition
}
```

#### **C++ Default Constructor**

A constructor which has no argument is known as default constructor. It is invoked at the time of creating object.

Let's see the simple example of C++ default Constructor.

```
#include <iostream>

using namespace std;

class Employee
{
    public:
        Employee()
        {
            cout<<"Default Constructor Invoked"<<endl;
        }
};

int main(void)
{
    Employee e1; //creating an object of Employee
    Employee e2;
    return 0;
}
```

### Output:

Default Constructor Invoked

Default Constructor Invoked

### C++ Parameterized Constructor

A constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects.

Let's see the simple example of C++ Parameterized Constructor.

```
#include <iostream.h>

using namespace std;

class Employee {
    public:
        int id;//data member (also instance variable)
        string name;//data member(also instance variable)
```

```
float salary;
Employee(int i, string n, float s)
{
    id = i;
    name = n;
    salary = s;
}
void display()
{
    cout<<id<<" "<<name<<" "<<salary<<endl;
}
};

int main(void) {
    Employee e1 =Employee(101, "Sonoo", 890000); //creating an object of Employee
    Employee e2=Employee(102, "Nakul", 59000);
    e1.display();
    e2.display();
    return 0;
}
```

**Output:**

```
101 Sonoo 890000
102 Nakul 59000
```

**Copy Constructor:**

A copy constructor is a member function that initializes an object using another object of the same class.

Syntax:

```
Sample(Sample &t)
{
    id=t.id;
}
```

**DESTRUCTORS:**

Destructor is another special member function that is called by the compiler when the scope of the object ends.

Syntax

```
~ Class_Name()

{

}
```

**Example for Constructor and destructor:**

```
//Sum of Digits in a number using constructor and destructor
#include<iostream.h>
#include<conio.h>
class DigitSum
{
public:
DigitSum(int n)
{
int sum=0,n1;
n1=n;
while(n!=0)
{
sum=sum+(n%10);
n/=10;
}
cout<<"Sum of all digit of"<<n1<<"is "<<sum<<endl;
}
~DigitSum(){
cout<<"Destructor complete the program and free up memory";
}
};
void main()
{
```

```
int num;  
clrscr();  
cout<<"Enter number:";  
cin>>num;  
DigitSum(num);  
getch();  
}
```

Output:

Enter number: 1245

Sum of all digits of 1245 is 12

Destructor complete the program and free up memory

### **NEW OPERATOR:**

The “new” operator in C++ is used to allocate memory dynamically for a variable or an object at runtime. This means that the memory is allocated during the execution of the program, as opposed to being allocated at compile time.

Syntax:

Pointer\_name=new datatype;

Or

pointer\_variable = new datatype(value);

Example:

```
int *ptr=new int;
```

or

```
int *ptr=new int(10);
```

Example:

```
#include <iostream>
```

```
int main() {
```

```
    int* pInt = new int; // dynamically allocate memory for an int
```

```
*pnt = 5;           // store the value 5 in the allocated memory

std::cout << *pnt;  // output the value stored in the allocated memory

delete pnt;         // deallocate the memory to prevent memory leak

return 0;

}
```

Output:

5

### DATA MEMBERS AND MEMBER FUNCTIONS:

The variables which are declared in any class by using any fundamental data types (like int, char, float etc) or derived data type (like class, structure, pointer etc.) are known as **Data Members**.

The functions which are declared either in private section or public section are known as **Member functions**.

There are two **types of data members/member functions in C++**:

1. Private members
2. Public members

#### 1) Private members

The members which are declared in private section of the class (using private access modifier) are known as private members. Private members can also be accessible within the same class in which they are declared.

#### 2) Public members

The members which are declared in public section of the class (using public access modifier) are known as public members.

### Example:

```
#include <iostream>
// class definition
class person {
```

```
// private data members
private:
char name[20];
int age;

// public member functions
Public:
void read(char n[], int a) {
    strcpy(name, n);
    age = a;
}

// function to print name and age
void display() {
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
}
};

// main function
void main() {
    // creating an object of person class
    person P;
    // calling member functions
    P.read("Priya", 27);
    P.display();
}
```

**Output:**

Priya  
27

**ACCESS SPEICIFIER / ACCESS MODIFIER:**

Access Modifiers or Access Specifiers in a class are used to assign the accessibility to the class members, i.e., they set some restrictions on the class members so that they can't be directly accessed by the outside functions.

There are 3 types of access modifiers available in C++:

1. Public
2. Private
3. Protected

**Public:**

All the class members declared under the public specifier will be available to everyone. The data members and member functions declared as public can be accessed by other classes and functions too.

Example:

```
include<iostream>
// class definition
class Circle
{
    public:
        double radius;

        double compute_area()
        {
            return 3.14*radius*radius;
        }
};

// main function
void main()
{
    Circle obj;

    // accessing public datamember outside class
    obj.radius = 5.5;

    cout << "Radius is: " << obj.radius << "\n";
    cout << "Area is: " << obj.compute_area();
}
```

Output:

Radius is: 5.5

Area is: 94.985

**Private:**

The class members declared as *private* can be accessed only by the member functions inside the class. They are not allowed to be accessed directly by any object or function outside the class. Only the member functions or the [friend functions](#) are allowed to access the private data members of the class.

Example:

```
#include<iostream>
class Circle
{
    // private data member
    private:
        double radius;

    // public member function
    public:
        void compute_area(double r)
        { // member function can access private
          // data member radius
            radius = r;

            double area = 3.14*radius*radius;

            cout << "Radius is: " << radius << endl;
            cout << "Area is: " << area;
        }
};

// main function
void main()
{
    // creating object of the class
    Circle obj;

    // trying to access private data member
    // directly outside the class
    obj.compute_area(1.5);
}
```

Output:

Radius is: 1.5  
Area is: 7.065

**Protected:**

The protected access modifier is similar to the private access modifier in the sense that it can't be accessed outside of its class unless with the help of a friend class. The difference is that the

class members declared as Protected can be accessed by any subclass (derived class) of that class as well.

Example:

```
#include <bits/stdc++.h>
// base class
class Parent
{
    // protected data members
    protected:
    int id_protected;

};

// sub class or derived class from public base class
class Child : public Parent
{
    public:
    void setId(int id)
    {
        // Child class is able to access the inherited
        // protected data members of base class

        id_protected = id;

    }
    void displayId()
    {
        cout << "id_protected is: " << id_protected << endl;
    }
};

// main function
void main() {

    Child obj1;
    obj1.setId(81);
    obj1.displayId();
    return 0;
}

Output:
id_protected is: 81
```

**SINGLETON CLASS:**

A singleton pattern is a design pattern that ensures that only one instance of a class can exist in the entire program. This means that if you try to create another instance of the class, it will return the same instance that was created earlier.

**FRIEND FUNCTION**

A friend function can access the private and protected data of a class. We declare a friend function using the friend keyword inside the body of the class.

Syntax:

```
friend returnType functionName(arguments);
```

Example:

```
#include <iostream>
class Distance {
    private:
        int meter;

        // friend function
        friend int addFive(Distance);

    public:
        Distance() : meter(2) {}
};

// friend function definition
int addFive(Distance d) {
    //accessing private members from the friend function
    d.meter += 5;
    return d.meter;
}

void main() {
    Distance D;
    cout << "Distance: " << addFive(D);
}
```

Output:

Distance:7

## FRIEND CLASS:

A **friend class** can access private and protected members of other classes in which it is declared as a friend. It is sometimes useful to allow a particular class to access private and protected members of other classes.

Example:

```
#include <iostream>
// forward declaration
class ClassB;

class ClassA {
    private:
        int numA;

        // friend class declaration
        friend class ClassB;

    public:
        // constructor to initialize numA to 12
        ClassA() : numA(12) {}
};

class ClassB {
    private:
        int numB;
    public:
        // constructor to initialize numB to 1
        ClassB() : numB(1) {}
        // member function to add numA
        // from ClassA and numB from ClassB
        int add() {
            ClassA objectA;
            return objectA.numA + numB;
        }
};

void main() {
    ClassB objectB;
    cout << "Sum: " << objectB.add();
}
Output:
Sum:13
```

**ARRAY OF OBJECTS:**

An array is the fundamental data structure which is collection of similar data items stored in contiguous memory under a common name. An array of objects in C++ is a sequential collection of objects of the same class type.

Example:

```
#include<iostream>
using namespace std;
class MyClass {
private:
    int data;
public:
    void initialize(int i){
        data = i;
    }
    void method(int i){
        cout<<"object "<<i+1<<endl;
    }
};
```

```
void main(){
    MyClass array_of_objects[5];
    for(int i=0; i<5; i++){
        array_of_objects[i].initialize(i+1);
    }
}
```

Output:

```
object 1
object 2
object 3
object 4
object 5
```

**DYNAMIC MEMORY ALLOCATION:**

Memory allocation is basically a way of booking a partial or entire part of system memory space for the execution of applications. Memory allocation may be executed through a method known as memory management.

There are two types of memory spaces in our device- static memory and dynamic memory.

1. **Static Memory:** In static memory allocation, memory is allotted and deallocated by the compiler on its very own.
2. **Dynamic Memory:** In dynamic allocation, the allocation and deallocation of the memory happen at runtime. That is, the memory must be allocated or de-allocated by the program all through the run-time of a C++ program. So the programmer is required to deallocate the dynamically allocated memory which is not in use.

Syntax for Dynamic Memory Allocation:

```
pointer_variable = new data_type[size];
```

Example:

```
#include <iostream>
void main() {
    int num;
    int *array{ new int[6]{ 1,2,3,4,5,6} };
    cout << "Array elements: " << endl;
    for (num = 0; num < 6; num++) {
        cout << array[num] << endl;
    }
}
```

Output:

Array elements:

```
1
2
3
4
5
6
```

**UNIT- II POLYMORPHISM****SYLLABUS:**

**Function Overloading:** Overloading a function - Default arguments – Overloading Constructors.

**Operator Overloading:** Overloading an operator as a member function – Overloading an operator as a friend function – Overloading the operators [], (), -> and comma operators – Conversion Functions.

**FUNCTIONS:**

A function is a block of code that performs a specific task. The components of function are: (1)Function Declaration, (2)Function Calling.

**Function Declaration:**

A function declaration tells the compiler about the number of parameters, data types of parameters, and returns type of function.

The syntax to declare a function is:

```
returnType functionName (parameter1, parameter2,...) {  
    // function body  
}
```

**Function Calling:**

A function calling is used to call the function from the main program.

Syntax:

```
function_name();
```

```
#include<iostream>  
  
void greet() {  
    // code  
}  
  
int main() {  
    ... ..  
    greet();  
    ... ..  
}
```

function call

**Example:**

```
// program to print a text
#include <iostream.h>
// display a number
void displayNum(int n1, float n2)           //Function declaration
{
    cout << "The int number is " << n1;
    cout << "The double number is " << n2;
}

void main() {
    int num1 = 5;
    double num2 = 5.5;

    // calling the function
    displayNum(num1, num2);
}
```

Output:

The int number is 5  
The double number is 5.5

**Types of Functions:**

There are two types of function:

1. **Standard Library Functions:** Predefined in C++
2. **User-defined Function:** Created by users

**Standard Library Functions:**

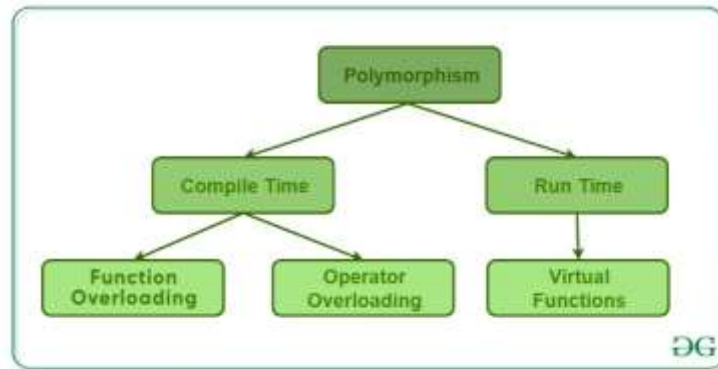
Library functions are the built-in functions in C++ programming. Programmers can use library functions by invoking the functions directly. Some common library functions in C++ are [sqrt\(\)](#), [abs\(\)](#), [isdigit\(\)](#), etc.

**User-defined Function:**

User-defined functions are user/customer-defined blocks of code specially customized to reduce the complexity of big programs.

**POLYMORPHISM:**

The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. A person at the same time can have different characteristics.

**Compile-Time Polymorphism**

This type of polymorphism is achieved by function overloading or operator overloading.

**FUNCTION OVERLOADING:**

Function overloading is using a single function name to perform different types of tasks. Polymorphism is extensively used in implementing inheritance.

**Example:**

```
#include <iostream.h>
using namespace std;
class Fun_overload {
public:
    void func(int x)
    {
        cout << "value of x is " << x << endl;
    }
    void func(double x)
    {
        cout << "value of x is " << x << endl;
    }
    void func(int x, int y)
    {
        cout << "value of x and y is " << x << ", " << y
```

```
        << endl;
    }
};

int main()
{
    fun_overload obj1;
    obj1.func(7);
    obj1.func(9.132);
    obj1.func(85, 64);
    return 0;
}
```

Output:

value of x is 7

value of x is 9.132

value of x and y is 85, 64

### **ARGUMENTS IN C++**

There are two types of arguments in C++ and they are:

- Default arguments
- Parameterized arguments
  - Pass by Reference
  - Pass by Value

### **DEFAULT ARGUMENTS**

When you write a function that takes some arguments as a parameter in C++, you can assign a default value to that argument while declaring it.

#### **Example:**

```
#include <iostream.h>

int SquareThisNum(int num = 2){
    return num * num;
}

int main() {
    // No argument is passed so square of 0 is returned.
    cout << SquareThisNum();
    return 0;
}
```

```
}
```

Output:

4

### PASS BY REFERENCE

In C++, **pass by reference** (also called pass by address) means, “pass the reference of an argument in the calling function to the corresponding formal parameter of the called function so that a copy of the address of the actual argument is made in memory,” i.e., the caller and the callee use the same variable for the parameter. If the callee modifies the parameter variable, then the effect is visible to the caller’s variable.

#### Example:

```
#include <iostream>
using namespace std;
void SquareThisNum(int num, int &result){// `result` argument in the callee
    result = num * num;
}
int main() {
    int res = 0;// caller function argument
    cout << res << endl;
    SquareThisNum(5, res);
    cout << res << endl;
}
```

Output:

0

25

### PASS BY VALUE

In C++, pass by value means that a copy of the actual argument’s value is made in the memory, i.e., the caller and callee have two independent variables with the same value. If the callee modifies the parameter value, the effect is not visible to the caller.

#### Example:

```
#include <iostream.h>
void SquareThisNum(int num, int result){
    result = num * num;
}
int main() {
```

```
int res = 0; // the caller function  
SquareThisNum(5, res);  
cout << res << endl;  
}
```

Output:

25

### **CONSTRUCTOR OVERLOADING:**

A constructor is a special member [function](#) that is called automatically when an [object](#) is created.

Constructors can be overloaded in a similar way as [function overloading](#). Overloaded constructors have the same name (name of the class) but the different number of arguments. Depending upon the number and type of arguments passed, the corresponding constructor is called.

Example:

*// C++ program to demonstrate constructor overloading*

```
#include <iostream.h>
```

```
class Person {
```

```
private:
```

```
int age;
```

```
public:
```

```
// 1. Constructor with no arguments
```

```
Person() {
```

```
    age = 20;
```

```
}
```

```
// 2. Constructor with an argument
```

```
Person(int a) {
```

```
    age = a;
```

```
}
```

```
int getAge() {
```

```
    return age;
```

```
}
```

```
};  
int main() {  
    Person person1, person2(45);  
  
    cout << "Person1 Age = " << person1.getAge() << endl;  
    cout << "Person2 Age = " << person2.getAge() << endl;  
    return 0;  
}
```

Output:

Person1 Age = 20

Person2 Age = 45

### **OPERATOR OVERLOADING:**

The process of making an operator exhibit different behaviors in different instances is known as operator overloading.

#### **Syntax:**

```
return_type class_name : : operator op(argument_list)  
{  
    // body of the function.  
}
```

#### **Example:**

```
#include <iostream.h>  
using namespace std;  
class Test  
{  
    private:  
        int num;  
    public:  
        Test(): num(8){}  
        void operator ++() {  
            num = num+2;  
        }  
        void Print() {
```

```
        cout<<"The Count is: "<<num;
    }
};

int main()
{
    Test tt;
    ++tt; // calling of a function "void operator ++()"
    tt.Print();
    return 0;
}
```

Output:

The Count is: 10

### **OVERLOADING THE OPERATOR WITH MEMBER FUNCTIONS:**

The Operator is handled as a class member when you overload an operator with a member function. According to this, a class member that defines the operator function must be one of the operands. Any compatible type, including additional user-defined types, may be used for the other operand.

Syntax:

```
return_type operator op(parameters) {
    // Define the behavior of the operator
    // You can access the current object using 'this' pointer
}
```

Example:

### **OVERLOADING THE OPERATOR WITH FRIEND FUNCTION:**

Another method for overloading operators with non-member functions designated as friends inside the class is friend functions. Although friend functions are not class members, they can access the class's private members.

Syntax:

```
friend return_type operator op(parameters);
```

Example:

```

#include <iostream>
class Complex {
Public:
    Complex(double real, double image) : real(real), imag(imag) {}
    // Declare the '+' operator as a friend
    friend Complex operator+(const Complex& c1, const Complex& c2);
    // Display the complex number
    void display() const {
        std::cout << real << " + " << imag << "i" << std::endl;
    }
Private:
    double real, imag;
};
// Define the '+' operator using a friend function
Complex operator+(const Complex& c1, const Complex& c2) {
    return Complex(c1.real + c2.real, c1.imag + c2.imag);
}
int main() {
    Complex c1(2.0, 3.0);
    Complex c2(1.5, 2.5);
    // Use the overloaded '+' operator
    Complex sum = c1 + c2;
    std::cout << "Sum: ";
    sum.display();
    return 0;
}

```

### **CONVERSION FUNCTION:**

Conversion function is defined as the function which converts from the type of its class to another specified type.

Syntax:

```

Operator datatype()
{
Return datatype;
}

```

Example:

```

#include <iostream.h>

```

```
class Time
{
int hrs,min,sec;
public:
Time() {hrs=min=sec=0;}

Time (int h, int m,int s)
{
hrs = h;
min = m;
sec = s;
}
operator int ()
{
int seconds;
seconds = hrs*60*60 + min*60 + sec;
return seconds;
}
};

int main()
{
int seconds;
Time t(2,15,30);
seconds = t;
std::cout<<seconds;// Will work
return 0;
}
Output:
8130
```

### UNIT III – INHERITANCE

#### SYLLABUS

**Inheritance:** Types of inheritance – *protected* access specifier – Virtual Base Class – Base class and derived class constructors. **Run-time Polymorphism:** Virtual Functions – Function overriding - Pure virtual function – Abstract base class

#### INHERITANCE:

Classes in C++ can be extended, creating new classes which retain characteristics of the base class. This process, known as inheritance, involves a *base class* and a *derived class*: The *derived class* inherits the members of the *base class*, on top of which it can add its own members.

#### Types of Inheritance:

- Single inheritance
- Multiple inheritance
- Hierarchical inheritance
- Multilevel inheritance
- Hybrid inheritance

#### Derived Class:

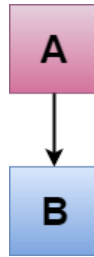
A Derived class is defined as the class derived from the base class.

Syntax:

```
class derived_class_name :: visibility-mode base_class_name
{
// body of the derived class.
}
```

#### Single Inheritance:

**Single inheritance** is defined as the inheritance in which a derived class is inherited from the only one base class.



Where 'A' is the base class, and 'B' is the derived class.

Example:

```
#include <iostream>

using namespace std;

class Account {
    public:
    float salary = 60000;
};

class Programmer: public Account {
    public:
    float bonus = 5000;
};

int main(void) {
    Programmer p1;
    cout<<"Salary: "<<p1.salary<<endl;
    cout<<"Bonus: "<<p1.bonus<<endl;
    return 0;
}
```

Output:

Salary: 60000

Bonus: 5000

### **Multilevel Inheritance:**

Multilevel inheritance is a process of deriving a class from another derived class.



```
#include <iostream>

using namespace std;

class Animal {
    public:
    void eat() {
        cout<<"Eating..."<<endl;
    }
};

class Dog: public Animal
{
    public:
    void bark(){
        cout<<"Barking..."<<endl;
    }
};

class BabyDog: public Dog
{
    public:
    void weep() {
        cout<<"Weeping...";
    }
};

int main(void) {
    BabyDog d1;
    d1.eat();
    d1.bark();
    d1.weep();
}
```

```
    return 0;
}
```

Output:

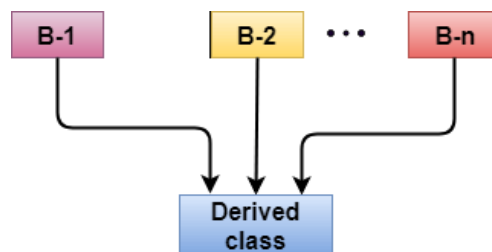
Eating...

Barking...

Weeping...

## Multiple Inheritance

**Multiple inheritance** is the process of deriving a new class that inherits the attributes from two or more classes.



### Syntax of the Derived class:

```
class D : visibility B-1, visibility B-2, ?
```

```
{
    // Body of the class;
}
```

Example of multiple inheritance.

```
#include <iostream>
```

```
using namespace std;
```

```
class A
```

```
{
    protected:
        int a;
    public:
        void get_a(int n)
        {
            a = n;
        }
}
```

```
};

class B
{
    protected:
    int b;
    public:
    void get_b(int n)
    {
        b = n;
    }
};

class C : public A, public B
{
    public:
    void display()
    {
        std::cout << "The value of a is : " << a << std::endl;
        std::cout << "The value of b is : " << b << std::endl;
        cout << "Addition of a and b is : " << a + b;
    }
};

int main()
{
    C c;
    c.get_a(10);
    c.get_b(20);
    c.display();

    return 0;
}
```

Output:

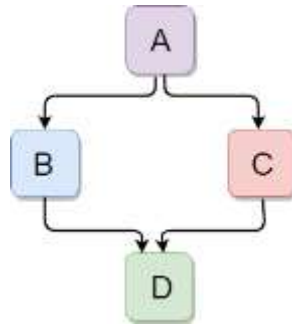
The value of a is : 10

The value of b is : 20

Addition of a and b is : 30

### Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance.



### VIRTUAL BASE CLASS

Virtual base classes in C++ are used to prevent multiple instances of a given class from appearing in an inheritance hierarchy when using multiple inheritances.

Virtual Class is defined by writing a keyword “virtual” in the derived classes, allowing only one copy of data to be copied to child classes.

It prevents multiple instances of a class appearing as a parent class in the inheritance hierarchy when multiple inheritances are used.

Let's see example without using virtual base class:

```
#include <iostream.h>
class A {
public:
    A() {
        cout << "Constructor A\n";
    }
    void display() {
        cout << "Hello form Class A \n";
    }
};

class B: public A {
```

```
};
class C: public A {
};
class D: public B, public C {
};
int main() {
    D object;
    object.display();
}
```

Output:

error: non-static member 'display' found in multiple base-class subobjects of type 'A':

```
class D -> class B -> class A
class D -> class C -> class A
object.display();
    ^
```

note: member found by ambiguous name lookup

To overcome this error, virtual base class is used.

Example using virtual base class:

```
#include <iostream.h>
class A {
public:
    A() {
        cout << "Constructor A\n";
    }
    void display() {
        cout << "Hello form Class A \n";
    }
};
```

```
class B: public virtual A {
};
class C: public virtual A {
};
class D: public B, public C {
};
int main() {
    D object;
    object.display();
}
```

Output:

Constructor A  
Hello from Class A

**BASE CLASS CONSTRUCTOR AND DERIVED CLASS CONSTRUCTOR:**

Example:

```
#include<iostream.h>
class A1
{
    public:
    A1()
    {
        cout << "Constructor of the base class A1 \n";
    }
};

class A2
{
    public:
    A2()
    {
        cout << "Constructor of the base class A2 \n";
    }
};

class S: public A1, virtual A2
{
    public:
    S(): A1(), A2()
    {
        cout << "Constructor of the derived class S \n";
    }
};

// Driver code
int main()
{
    S obj;
    return 0;
}
Output:
```

Constructor of the base class A2  
Constructor of the base class A1  
Constructor of the derived class S

**VIRTUAL FUNCTIONS:**

A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword. It is used to tell the compiler to perform dynamic linkage or late binding on the function.

```
#include <iostream>

{
public:
virtual void display()
{
    cout << "Base class is invoked"<<endl;
}
};

class B:public A
{
public:
void display()
{
    cout << "Derived Class is invoked"<<endl;
}
};

int main()
{
    A* a; //pointer of base class
    B b;  //object of derived class
    a = &b;
    a->display(); //Late Binding occurs
}
```

**Output:**

Derived Class is invoked

**FUNCTION OVERRIDING**

If derived class defines same function as defined in its base class, it is known as function overriding in C++. It is used to achieve runtime polymorphism. It enables you to provide specific implementation of the function which is already provided by its base class.

Example:

```
#include <iostream.h>
class Animal {
    public:
    void eat(){
        cout<<"Eating...";
    }
};
class Dog: public Animal
{
    public:
    void eat()
    {
        cout<<"Eating bread...";
    }
};
int main(void) {
    Dog d = Dog();
    d.eat();
    return 0;
}
Output:
Eating bread...
```

**ABSTRACT CLASS:**

An *abstract class* is a class that is designed to be specifically used as a base class. An abstract class contains at least one *pure virtual function*. You declare a pure virtual function by using a *pure specifier* (= 0) in the declaration of a virtual member function in the class declaration.

Example:

```
class AB {
    public:
    virtual void f() = 0;
};
```

**PURE VIRTUAL FUNCTIONS:**

A pure virtual function is a virtual function that has no definition within the class. It can be considered as an empty function means that the pure virtual function does not have any definition relative to the base class.

Programmers need to redefine the pure virtual function in the derived class as it has no definition in the base class.

Syntax:

Virtual void display() = 0; or virtual void display()= {};

Example:

```
#include <iostream.h>
class Base {
    int x;
public:
    //pure virtual function
    virtual void fun() = 0;
    int getX() { return x; }
};
class Derived : public Base {
    int y;
public:
    // implementation of the pure virtual function
    void fun() { cout << "fun() called"; }
};

int main(void)
{
    Derived d;
    // calling the fun() function of Derived class
    d.fun();
    return 0;
}
```

Output:

Fun() called

**UNIT IV – FILES & TEMPLATES****SYLLABUS**

**Templates:** Function templates – Overloading a function template – Class templates.

**Standard Template Library (STL):** Containers: vector, list – Iterators: forward, backward – Algorithms: removing and replacing elements, sorting, counting, reversing a sequence.

**Exception Handling:** Exceptions – try, catch, throw – Rethrowing an exception – Restricting exceptions - Handling exceptions in derived classes - terminate(), abort(), unexpected(), set\_terminate().

**TEMPLATES:**

A C++ template is a powerful feature added to C++. It allows you to define the generic classes and generic functions and thus provides support for generic programming. The template can be represented in two ways:

- Function templates
- Class templates

**FUNCTION TEMPLATES**

Templates are powerful features of C++ which allows us to write generic programs. We can create a single function to work with different data types by using a template.

**Defining a Function Template**

A function template starts with the keyword template followed by template parameter(s) inside <> which is followed by the function definition.

```
template <typename T>
T functionName(T parameter1, T parameter2, ...) {
    // code
}
```

**Calling a Function Template**

Once we've declared and defined a function template, we can call it in other functions or templates (such as the main() function) with the following syntax:

```
functionName<dataType>(parameter1, parameter2,...);
```

**Example:**

```
#include <iostream.h>

template <typename T>
T add(T num1, T num2) {
    return (num1 + num2);
}

int main() {
    int result1;
    double result2;
    // calling with int parameters
    result1 = add<int>(2, 3);
    cout << "2 + 3 = " << result1 << endl;

    // calling with double parameters
    result2 = add<double>(2.2, 3.3);
    cout << "2.2 + 3.3 = " << result2 << endl;

    return 0;
}
```

Output:

2 + 3 = 5

2.2 + 3.3 = 5.5

## **OVERLOADING A FUNCTION TEMPLATE**

The function may have the same definition, but with different arguments is known as function overloading.

The name of the function templates are the same but called with different arguments is known as **function template overloading**.

Example:

```
#include <iostream.h>
// Template declaration
template <class T>

// Template overloading of function
void display(T t1)
{
    cout << "Displaying Template: "
        << t1 << "\n";
}
```

```

}

// Template overloading of function
void display(int t1)
{
    cout << "Explicitly display: "
        << t1 << "\n";
}

// Driver Code
int main()
{
    // Function Call with a
    // different arguments
    display(200);
    display(12.40);
    display('G');

    return 0;
}

```

Output:

Explicitly display: 200

Displaying Template: 12.4

Displaying Template: G

## **CLASS TEMPLATE**

Class Template can also be defined similarly to the Function Template. When a class uses the concept of Template, then the class is known as generic class.

Syntax:

```

template<class Ttype>
class class_name
{
    .
    .
}

```

**Example:**

```

#include <iostream.h>
template<class T>
class A

```

```
{
    public:
        T num1 = 5;
        T num2 = 6;
        void add()
        {
            std::cout << "Addition of num1 and num2 : " << num1+num2<<std::endl;
        }
};

int main()
{
    A<int> d;
    d.add();
    return 0;
}
```

Output:

Addition of num1 and num2 : 11

### **STANDARD TEMPLATE LIBRARY:**

The Standard Template Library (STL) is a set of C++ template classes to provide common programming data structures and functions such as lists, stacks, arrays, etc.

The C++ Standard Template Library (STL) is a collection of algorithms, data structures, and other components that can be used to simplify the development of C++ programs.

STL has 4 components:

- Algorithms
- Containers
- Functors
- Iterators

### **Algorithms**

The header algorithm defines a collection of functions specially designed to be used on a range of elements. They act on containers and provide means for various operations for the contents of the containers.

- Algorithm
  - Sorting
  - Searching
  - Important STL Algorithms
  - Useful Array algorithms
  - Partition Operations
- Numeric
  - valarray class

### Containers

Containers or container classes store objects and data. There are in total seven standards “first-class” container classes and three container adaptor classes and only seven header files that provide access to these containers or container adaptors.

- Sequence Containers: implement data structures that can be accessed in a sequential manner.
  - vector
  - list
  - deque
  - arrays
  - forward\_list( Introduced in C++11)
- Container Adaptors: provide a different interface for sequential containers.
  - queue
  - priority\_queue
  - stack
- Associative Containers: implement sorted data structures that can be quickly searched ( $O(\log n)$  complexity).

- set
- multiset
- map
- multimap
- Unordered Associative Containers: implement unordered data structures that can be quickly searched
  - unordered\_set (Introduced in C++11)
  - unordered\_multiset (Introduced in C++11)
  - unordered\_map (Introduced in C++11)
  - unordered\_multimap (Introduced in C++11)

### VECTORS:

In C++, vectors are used to store elements of similar data types. However, unlike [arrays](#), the size of a vector can grow dynamically.

That is, we can change the size of the vector during the execution of a program as per our requirements.

Vectors are part of the [C++ Standard Template Library](#). To use vectors, we need to include the vector header file in our program.

Example:

```
#include <iostream.h>
#include <vector>

int main() {

    // initializer list
    vector<int> vector1 = {1, 2, 3, 4, 5};

    // uniform initialization
    vector<int> vector2{6, 7, 8, 9, 10};

    // method 3
    vector<int> vector3(5, 12);
```

```
cout << "vector1 = ";

// ranged loop
for (const int& i : vector1) {
    cout << i << " ";
}

cout << "\nvector2 = ";

// ranged loop
for (const int& i : vector2) {
    cout << i << " ";
}
```

Output:

```
vector1 = 1 2 3 4 5
vector2 = 6 7 8 9 10
vector3 = 12 12 12 12 12
```

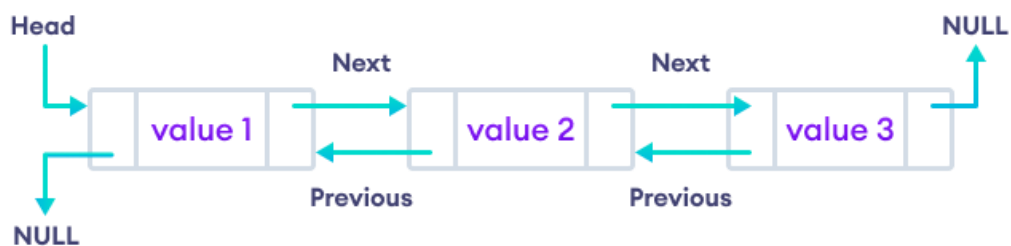
### Basic Operations on Vector:

- Add elements
- Access elements
- Change elements
- Remove elements

### LIST:

C++ List is a [STL container](#) that stores elements randomly in unrelated locations. To maintain sequential ordering, every list element includes two links:

- one that points to the previous element
- another that points to the next element



**Example:**

```
#include <iostream>
#include <list>
int main() {

    // create the list
    list<int> numbers {1, 2, 3, 4};

    // display the elements of the list
    cout << "List Elements: ";
    for(int number : numbers) {
        cout << number << ", ";
    }
    return 0;
}
```

Output:

List Elements: 1, 2, 3, 4,

### Basic Operations on List:

1. Add elements
2. Access elements
3. Remove elements

### ALGORITHMS

Algorithms are the functions used across a variety of containers for processing its contents.

STL Algorithms can be categorized into:

- Mutating Algorithm
  - Non-mutating Algorithm
  - Sorting algorithm
  - Set algorithm
  - Relational algorithm
- **Nonmutating algorithms:** Nonmutating algorithms are the algorithms that do not alter any value of a container object nor do they change the order of the elements in which they appear. These algorithms can be used for all the container objects, and they make use of the forward iterators.
- **Mutating algorithms:** Mutating algorithms are the algorithms that can be used to alter the value of a container. They can also be used to change the order of the elements in which they appear.

- **Sorting algorithms:** Sorting algorithms are the modifying algorithms used to sort the elements in a container.
- **Set algorithms:** Set algorithms are also known as sorted range algorithm. This algorithm is used to perform some function on a container that greatly improves the efficiency of a program.
- **Relational algorithms:** Relational algorithms are the algorithms used to work on the numerical data. They are mainly designed to perform the mathematical operations to all the elements in a container.

### Functors

The STL includes classes that overload the function call operator. Instances of such classes are called function objects or functors. Functors allow the working of the associated function to be customized with the help of parameters to be passed. Must Read – Functors

### Iterators

Iterators are objects that are used to access elements of a container.

We can declare an iterator for each container in the C++ Standard Template Library. For example,

```
vector<int>::iterator it;
```

We often use iterator member functions like `begin()`, `end()`, etc. to return iterators that point to container elements. For example,

```
vector<int> numbers = {3, 2, 5, 1, 4};
```

```
vector<int>::iterator itr1 = numbers.begin();
```

```
vector<int>::iterator itr2 = numbers.end();
```

Example:

```
#include <iostream.h>
```

```
#include <vector>
```

```
int main() {
```

```
    // initialize vector of int type
```

```
    vector<int> numbers {1, 2, 3, 4, 5};
```

```
// initialize vector iterator to point to the first element
vector<int>::iterator itr = numbers.begin();
cout << "First Element: " << *itr << " "<<endl;

// change iterator to point to the last element
itr = numbers.end() - 1;
cout << "Last Element: " << *itr;
return 0;
}
```

Output:

First Element: 1

Last Element: 5

## **EXCEPTION HANDLING**

In C++, exceptions are runtime anomalies or abnormal conditions that a program encounters during its execution. The process of handling these exceptions is called exception handling.

### **Types of C++ Exception**

There are two types of exceptions in C++

1. **Synchronous:** Exceptions that happen when something goes wrong because of a mistake in the input data or when the program is not equipped to handle the current type of data it's working with, such as dividing a number by zero.
2. **Asynchronous:** Exceptions that are beyond the program's control, such as disc failure, keyboard interrupts, etc.

C++ provides an inbuilt feature for Exception Handling. It can be done using the following specialized keywords: try, catch, and throw with each having a different purpose.

Syntax of try-catch in C++

```
try {
    // Code that might throw an exception
    throw SomeExceptionType("Error message");
}
catch( ExceptionName e1 ) {
    // catch block catches the exception that is thrown from try block
```

}

### 1. try in C++

The try keyword represents a block of code that may throw an exception placed inside the try block. It's followed by one or more catch blocks. If an exception occurs, try block throws that exception.

### 2. catch in C++

The catch statement represents a block of code that is executed when a particular exception is thrown from the try block. The code to handle the exception is written inside the catch block.

```
#include <iostream>
using namespace std;
float division(int x, int y) {
    if( y == 0 ) {
        throw "Attempted to divide by zero!";
    }
    return (x/y);
}
int main () {
    int i = 25;
    int j = 0;
    float k = 0;
    try {
        k = division(i, j);
        cout << k << endl;
    } catch (const char* e) {
        cerr << e << endl;
    }
    return 0;
}
```

Output:

Attempted to divide by zero!

### 3. throw in C++

An exception in C++ can be thrown using the throw keyword. When a program encounters a throw statement, then it immediately terminates the current function and starts finding a matching catch block to handle the thrown exception.

**UNIT V****SYLLABUS**

**I/O Streams:** Formatted I/O with ios class functions - Manipulators – Creating own manipulator – Overloading << and >> operators.

**File I/O:** fstream class – Opening and closing a file – Reading from and writing to a text file - Unformatted and Binary I/O – Random access I/O.

**I/O STREAM:**

To perform any input and output operations in C++, we need to use iostream header files. Without an <iostream> header file, we cannot take input from the user or print any output

Syntax:

```
#include <iostream.h>
```

There are two types of streams in the iostream header file

**1. Input Stream:** To take any input from the user, we need to use cin, which belongs to the input stream

**Syntax to use input stream:**

```
1. std::cin>>variable_name
```

When the cin is executed, the cursor will be stopped at the particular statement until the value is entered. The value entered will be stored in a variable.

**2. Output Stream:** To print the output, we use built-in functions in the cout output stream

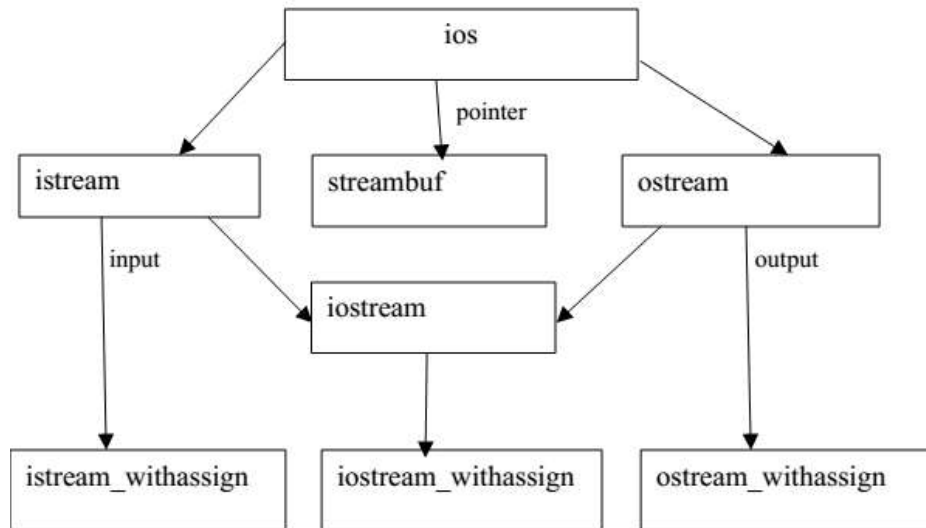
**Syntax for cout**

```
1. std::cout<<variable_name
```

When cout is executed, the value in the variable will be printed.

**STREAM CLASSES:**

C++ has number of classes that work with console and file operations. These classes are known as stream classes..



Stream classes for console I/O operations

### **FORMATTED AND UNFORMATTED DATA:**

Formatting means representation of data with different settings as per the requirement of the user. The various settings that can be done are number format, field width, decimal points, etc.

The data accepted or printed with default setting by the I/O function of the language is known as unformatted data. For example, cin, cout.

If the user needs to accept or display data in hexadecimal format, manipulators with I/O functions should be used. The data obtained or represented or represented with these manipulators are known as formatted data. For example, `cout<<hex<<15;`

### **FORMATTED CONSOLE I/O OPERATIONS:**

There are three types of formatting the output.

- IOS class function and flags
- Manipulators
- User-defined output functions

#### **IOS Class Function and Flags:**

The ios grants operations common to both input and output. The iostream class is automatically loaded in the program by the compiler.

- istream performs formatted input

- ostream performs formatted output
- istream performs formatted input and output

| FUNCTION    | WORKING                                                                       |
|-------------|-------------------------------------------------------------------------------|
| width()     | To set the required field width. The output will be displayed in given width. |
| precision() | To set number of decimal points to a float value.                             |
| fill()      | To set a character to fill in the blank space of a field.                     |
| setf()      | To set various flags for formatting output.                                   |
| unsetf()    | To remove the flag setting.                                                   |

Example:

```
#include<iostream.h>
#include<conio.h>
int main(){
clrscr();
cout.width(5);
cout<<"A";
cout.width(15);
cout<<"B";
return 0;
}
```

Output:

A    B

### Manipulators

**Manipulators** are helping functions that can modify the [input/output](#) stream. It does not mean that we change the value of a variable, it only modifies the I/O stream using insertion (<<) and extraction (>>) operators. For example, to print hexadecimal value of 100 is as follows:

```
cout<<setbase(16)<<100;
```

| FUNCTION    | WORKING                           |
|-------------|-----------------------------------|
|             |                                   |
| setw(int n) | The field width is fixed to n     |
| setbase     | Set the base of the number system |

|                     |                                                                 |
|---------------------|-----------------------------------------------------------------|
| setprecision(int p) | The precision point is fixed to p                               |
| setfill(char f)     | The fill character is set to the character stored in variable f |
| endl                | Splits a new line                                               |
| ends                | Add null character to close an output string                    |
| flush               | Flushes the buffer stream                                       |
| lock                | Locks the file associated with the file handle                  |
| hex, oct, dec       | Displays the number in hexadecimal, octal and decimal format    |

Example:

Write a program to read a number in hexadecimal format using cin statement and display the number in decimal format.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void main()
{
int x;
clrscr();
cout<<"Enter Hexadecimal value:";
cin>>hex>>x;
cout<<"The decimal number is:";
cout<<dec<<x;
getch();
}
```

Output:

Enter Hexadecimal value:100

The decimal number is: 256

### **UNFORMATTED I/O OPERATIONS:**

The input stream uses cin object to read data and the output stream uses cout object to display the data on the screen. The cin and cout are pre-defined streams for input and output data. The datatype is identified by these functions using operator overloading of the operators << (insertion operator) and >> (extraction operator).

Input stream – It reads operation through keyboard. It uses cin as object. The cin statement uses >> before a variable name.

Output Stream – it manage output of the stream, that is, display contents of variables on the screen. It uses << before the variable name. it uses the cout as object.

Example:

```
#include<iostream.h>
#include<conio.h>
void main()
{
int weight, height;
clrscr();
cin>>weight>>height;
cout<<"The values are€:";
cout<<weight<<height;
getch();
}
20
40
The values are: 20
40
```

### OVERLOADING << AND >> OPERATORS

In C++, Input and Output are performed as a sequence of bytes, also known as streams. [Input and Output stream](#) are managed by the iostream library. **cin** and **cout** are the standard objects for the input stream and output stream.

Example:

```
#include <iostream>
using namespace std;

class Distance {
private:
    int feet;        // 0 to infinite
    int inches;      // 0 to 12

public:
    // required constructors
    Distance() {
        feet = 0;
        inches = 0;
    }
    Distance(int f, int i) {
        feet = f;
```

```

        inches = i;
    }
    friend ostream &operator<<( ostream &output, const Distance &D ) {
        output << "F : " << D.feet << " I : " << D.inches;
        return output;
    }

    friend istream &operator>>( istream &input, Distance &D ) {
        input >> D.feet >> D.inches;
        return input;
    }
};

int main() {
    Distance D1(11, 10), D2(5, 11), D3;

    cout << "Enter the value of object : " << endl;
    cin >> D3;
    cout << "First Distance : " << D1 << endl;
    cout << "Second Distance : " << D2 << endl;
    cout << "Third Distance : " << D3 << endl;

    return 0;
}

```

Output:

\$/a.out

Enter the value of object :

70

10

First Distance : F : 11 I : 10

Second Distance : F : 5 I : 11

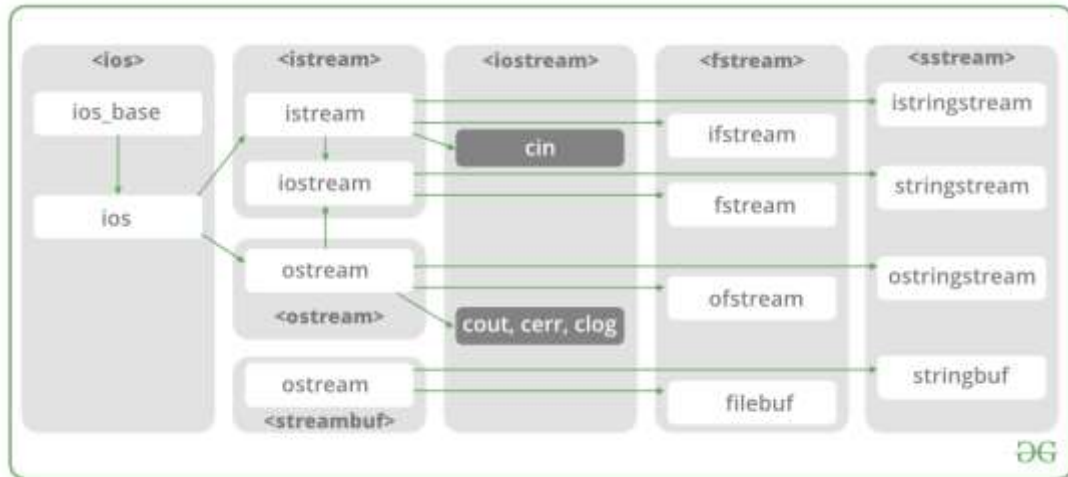
Third Distance : F : 70 I : 10

## 4.2 FILES

To read and write from a file we are using the standard C++ library called fstream. Let us see the data types define in fstream library is:

| Data Type | Description                                                                              |
|-----------|------------------------------------------------------------------------------------------|
| fstream   | It is used to create files, write information to files, and read information from files. |

|          |                                                                |
|----------|----------------------------------------------------------------|
| ifstream | It is used to read information from files.                     |
| ofstream | It is used to create files and write information to the files. |



### CLASSES OF FILE STREAMS:

#### 1. ios:-

ios stands for input output stream.

- This class is the base class for other classes in this class hierarchy.
- This class contains the necessary facilities that are used by all the other derived classes for input and output operations.

#### 2. istream:-

- istream stands for input stream.
- This class is derived from the class 'ios'.
- This class handle input stream.
- The extraction operator(>>) is overloaded in this class to handle input streams from files to the program execution.
- This class declares input functions such as get(), getline() and read().

#### 3. ostream:-

- ostream stands for output stream.
- This class is derived from the class 'ios'.
- This class handle output stream.
- The insertion operator(<<) is overloaded in this class to handle output streams to files from the program execution.
- This class declares output functions such as put() and write().

### 4. stringstream:-

- This class contains a pointer which points to the buffer which is used to manage the input and output streams.

### 5. fstreambase:-

- This class provides operations common to the file streams. Serves as a base for fstream, ifstream and ofstream class.
- This class contains open() and close() function.

### 6. ifstream:-

- This class provides input operations.
- It contains open() function with default input mode.
- Inherits the functions get(), getline(), read(), seekg() and tellg() functions from the istream.

### 7. ofstream:-

- This class provides output operations.
- It contains open() function with default output mode.
- Inherits the functions put(), write(), seekp() and tellp() functions from the ostream.

### 8. fstream:-

- This class provides support for simultaneous input and output operations.
- Inherits all the functions from istream and ostream classes through iostream.

**9. filebuf:-**

- Its purpose is to set the file buffers to read and write.
- We can also use file buffer member function to determine the length of the file.

**C++ FileStream example: writing to a file:**

Let's see the simple example of writing to a text file testout.txt using C++ FileStream programming.

```
#include <iostream>
#include <fstream>
using namespace std;
int main () {
    ofstream filestream("testout.txt");
    if (filestream.is_open())
    {
        filestream << "Welcome to javaTpoint.\n";
        filestream << "C++ Tutorial.\n";
        filestream.close();
    }
    else cout << "File opening is fail.";
    return 0;
}
```

Output:

**C++ FileStream example: reading from a file:**

Let's see the simple example of reading from a text file testout.txt using C++ FileStream programming.

```
#include <iostream>
#include <fstream>
using namespace std;
int main () {
```

```
string srg;
ifstream filestream("testout.txt");
if (filestream.is_open())
{
    while ( getline (filestream,srg) )
    {
        cout << srg <<endl;
    }
    filestream.close();
}
else {
    cout << "File opening is fail."<<endl;
}
return 0;
}
```

Output:

Welcome to javaTpoint.

### **C++ Read and Write Example**

Let's see the simple example of writing the data to a text file testout.txt and then reading the data from the file using C++ FileStream programming.

```
#include <fstream>
#include <iostream>
using namespace std;
int main () {
    char input[75];
    ofstream os;
    os.open("testout.txt");
    cout <<"Writing to a text file:" << endl;
    cout << "Please Enter your name: ";
    cin.getline(input, 100);
    os << input << endl;
    cout << "Please Enter your age: ";
```

```
cin >> input;
cin.ignore();
os << input << endl;
os.close();
ifstream is;
string line;
is.open("testout.txt");
cout << "Reading from a text file:" << endl;
while (getline (is,line))
{
    cout << line << endl;
}
is.close();
return 0;
}
```

Output:

Writing to a text file:

Please Enter your name: Nakul Jain

Please Enter your age: 22

Reading from a text file: Nakul Jain

22